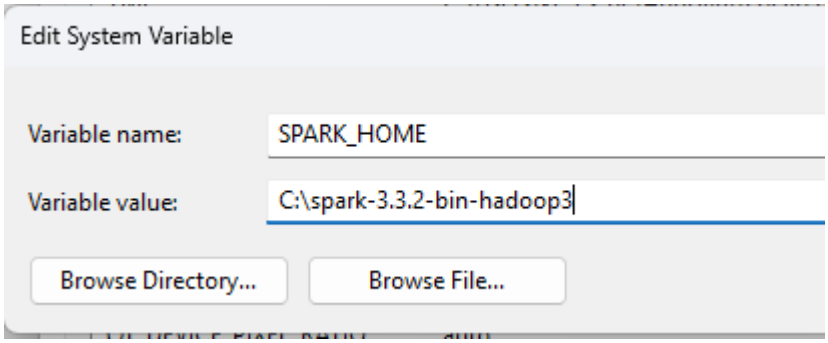
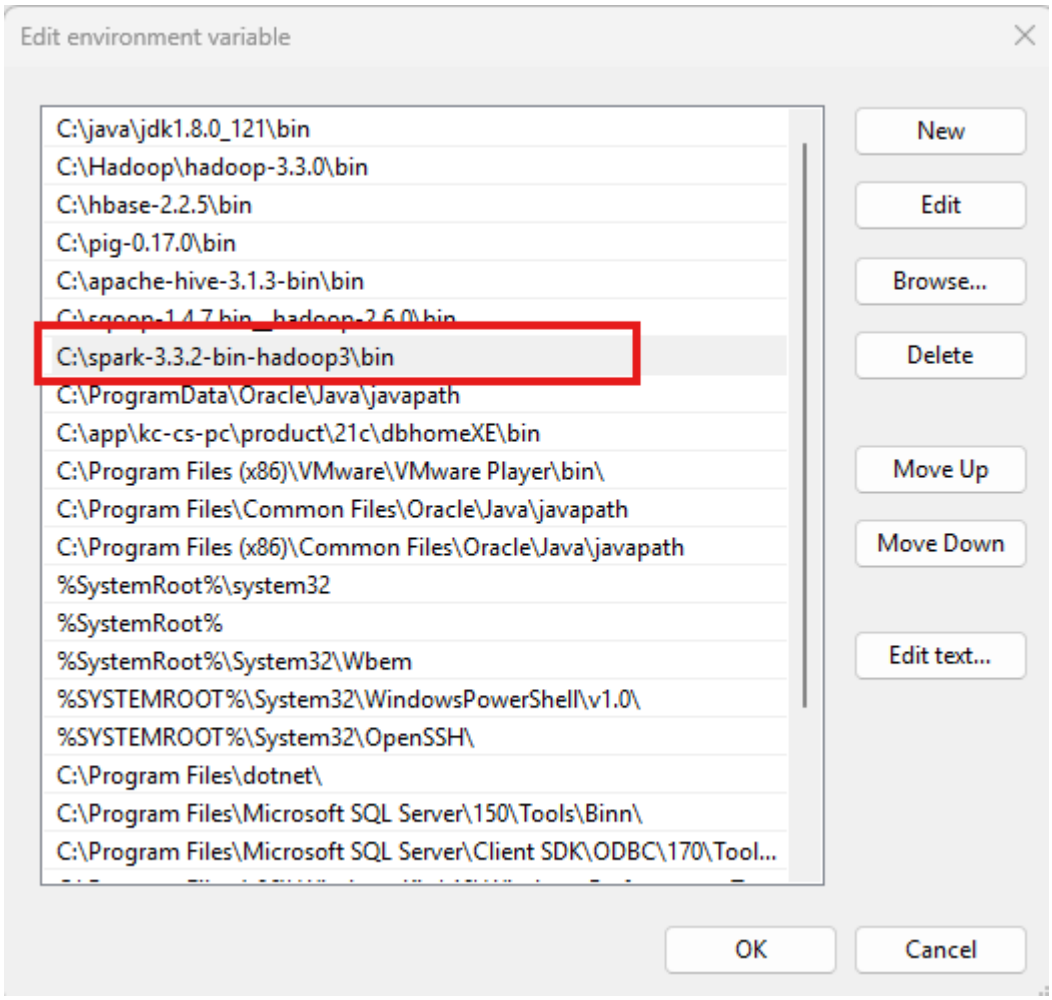
PRACTICAL-7 : SPARK

1. Download the **spark-3.3.2-bin-hadoop3.tgz** from
https://www.apache.org/dyn/closer.lua/spark/spark-3.3.2/spark-3.3.2-bin
hadoop3.tgz
2. Extract the **file into C:\ drive.**
3. Add the **SPARK_HOME variable** with:



4. Add the Path variable with:



5. Now run the spark-shell command in the cmd terminal:

```
C:\spark-3.3.2-bin-hadoop3>spark-shell
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
Spark context Web UI available at http://CP-CS-PC38:4040
Spark context available as 'sc' (master = local[*], app id = local-1775811965444).
Spark session available as 'spark'.
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /___/ .__/\_,_/_/ /_/\_\   version 3.3.2
      /_/

Using Scala version 2.12.15 (Java HotSpot(TM) 64-Bit Server VM, Java 1.8.0_121)
Type in expressions to have them evaluated.
Type :help for more information.
```

6. Basic command execution testing:

a. Spark.version

b. Print("hello spark")

```
scala> spark26/04/10 14:36:17 WARN
.version
res0: String = 3.3.2

scala> print("hello spark")
hello spark
scala>
```

Spark Installed successfully

Wordcount

1. **val data=sc.textFile("sparkdata.txt");**

```
scala> val data=sc.textFile("sparkdata.txt");
data: org.apache.spark.rdd.RDD[String] = sparkdata.txt MapPartitionsRDD[1] at textFile at <console>:23
```

2. **data.collect;**

```
scala> data.collect;
res2: Array[String] = Array(The ICC Men's T20 World Cup 2026 is currently in its final stages.,
lude India, England, South Africa, and New Zealand., "", India faced England in a high-voltage
rnament has recorded massive global viewership and digital engagement., "", Several batting and
```

3. **val splitdata = data.flatMap(line => line.split(" "));**

```
scala> val splitdata = data.flatMap(line => line.split(" "));
splitdata: org.apache.spark.rdd.RDD[String] = MapPartitionsRDD[2] at flatMap at <console>:23
```

4. **splitdata.collect;**

```
scala> splitdata.collect;
res3: Array[String] = Array(The, ICC, Men's, T20, World, Cup, 2026, is, currently, in, its, final, stages.,
unds., "", Semi-finalists, include, India,, England,, South, Africa,, and, New, Zealand., "", India, faced,
 final, is, scheduled, in, early, March, 2026., "", The, tournament, has, recorded, massive, global, viewer
2026, is, considered, one, of, the, most, competitive, T20, World, Cups, so, far.)
```

5. **val mapdata = splitdata.map(word =>(word,1));**

```
scala> val mapdata = splitdata.map(word =>(word,1));
mapdata: org.apache.spark.rdd.RDD[(String, Int)] = MapPartitionsRDD[3] at map at <console>:23
```

6. **mapdata.collect;**

```
scala> mapdata.collect;
res4: Array[(String, Int)] = Array((The,1), (ICC,1), (Men's,1), (T20,1), (World,1), (Cup,1), (2026,1), (is,1),
,1), (and,1), (Sri,1), (Lanka.,1), ("",1), (20,1), (teams,1), (competed,1), (across,1), (Group,1), (Stage,1),
1), (and,1), (New,1), (Zealand.,1), ("",1), (India,1), (faced,1), (England,1), (in,1), (a,1), (high-voltage,1)
mi-final.,1), ("",1), (The,1), (final,1), (is,1), (scheduled,1), (in,1), (early,1), (M...
```

7. **val reducedata = mapdata.reduceByKey(_+_);**

```
scala> val reducedata = mapdata.reduceByKey(_+_);
reducedata: org.apache.spark.rdd.RDD[(String, Int)] = ShuffledRDD[4] at reduceByKey at <console>:23
```

8. **reducedata.collect;**

```
scala> reducedata.collect;
res5: Array[(String, Int)] = Array((Africa,1), (New,2), (records,1), (8,1), (this,1), (teams,1), (2026.,1), (is,4),
ewership,1), (one,1), (T20,2), ("",9), (been,1), (massive,1), (World,2), (recorded,1), (Semi-finalists,1), (bowling,
nd,1), (high-voltage,1), (batting,1), (South,2), (The,4), (competed,1), (March,1), (most,1), (a,1), (challenged,1),
1), (played,1), (by,1), (Zealand.,1), (rounds.,1), (competitive,1), (scheduled,1), (st...
```

RDD shared variables

1. **val info = Array(1, 2, 3, 4)**

   val distinfo = sc.parallelize(info)

```
scala> val info = Array(1, 2, 3, 4)
info: Array[Int] = Array(1, 2, 3, 4)

scala> val distinfo = sc.parallelize(info)
distinfo: org.apache.spark.rdd.RDD[Int] = ParallelCollectionRDD[5] at parallelize at <console>:24
```

2. **val v = sc.broadcast(Array(1, 2, 3))**

   v.value

```
scala> val v = sc.broadcast(Array(1, 2, 3))
v: org.apache.spark.broadcast.Broadcast[Array[Int]] = Broadcast(6)

scala> v.value
res4: Array[Int] = Array(1, 2, 3)
```

3. **val a=sc.longAccumulator("Accumulator")**

   sc.parallelize(Array(2,5)).foreach(x=>a.add(x))

   a.value

```
scala> val a=sc.longAccumulator("Accumulator")
a: org.apache.spark.util.LongAccumulator = LongAccumulator(id: 175, name: Some(Accumulator), value: 0)

scala> sc.parallelize(Array(2,5)).foreach(x=>a.add(x))

scala> a.value
res6: Long = 7
```

Map function

val data1 = sc.parallelize(List(10,20,30))

data1.collect;

val mapfunc = data1.map(x => x+10)

mapfunc.collect

```
scala> val data1 = sc.parallelize(List(10,20,30))
data1: org.apache.spark.rdd.RDD[Int] = ParallelCollectionRDD[7] at parallelize at <console>:23

scala> data1.collect;
res9: Array[Int] = Array(10, 20, 30)

scala> val mapfunc = data1.map(x => x+10)
mapfunc: org.apache.spark.rdd.RDD[Int] = MapPartitionsRDD[8] at map at <console>:23

scala> mapfunc.collect
res10: Array[Int] = Array(20, 30, 40)
```

Filter function

val data2 = sc.parallelize(List(10,20,35,40))

data2.collect;

val filterfunc = data2.filter(x => x!=35)

filterfunc.collect;

```
scala> val data2 = sc.parallelize(List(10,20,35,40))
data2: org.apache.spark.rdd.RDD[Int] = ParallelCollectionRDD[9] at parallelize at <console>:23

scala> data2.collect;
res11: Array[Int] = Array(10, 20, 35, 40)

scala> val filterfunc = data2.filter(x => x!=35)
filterfunc: org.apache.spark.rdd.RDD[Int] = MapPartitionsRDD[10] at filter at <console>:23

scala> filterfunc.collect;
res12: Array[Int] = Array(10, 20, 40)
```

Count function

val data = sc.parallelize(List(1,2,3,4,5))

data.collect

val countfunc = data.count()

```
scala> val data = sc.parallelize(List(1,2,3,4,5))
data: org.apache.spark.rdd.RDD[Int] = ParallelCollectionRDD[11] at parallelize at <console>:23

scala> data.collect
res13: Array[Int] = Array(1, 2, 3, 4, 5)

scala> val countfunc = data.count()
countfunc: Long = 5
```

Distinct function

val data1 = sc.parallelize(List(10,20,20,40))

data1.collect

val distinctfunc = data1.distinct()

distinctfunc.collect

```
scala> val data1 = sc.parallelize(List(10,20,20,40))
data1: org.apache.spark.rdd.RDD[Int] = ParallelCollectionRDD[12] at parallelize at <console>:23

scala> data1.collect
res14: Array[Int] = Array(10, 20, 20, 40)

scala> val distinctfunc = data1.distinct()
distinctfunc: org.apache.spark.rdd.RDD[Int] = MapPartitionsRDD[15] at distinct at <console>:23

scala> distinctfunc.collect
res15: Array[Int] = Array(20, 40, 10)
```

Union function

val unionfunc = data.union(data1)

unionfunc.collect

```
scala> val unionfunc = data.union(data1)
unionfunc: org.apache.spark.rdd.RDD[Int] = UnionRDD[16] at union at <console>:24

scala> unionfunc.collect
res16: Array[Int] = Array(1, 2, 3, 4, 5, 10, 20, 20, 40)
```

Intersection tool

val data2 = sc.parallelize(List(1,20,40))

data2.collect

val intersectfunc = data1.intersection(data2)

intersecfunc.collect

```
scala>  val data2 = sc.parallelize(List(1,20,40))
data2: org.apache.spark.rdd.RDD[Int] = ParallelCollectionRDD[6] at parallelize at <console>:23

scala> data2.collect
res4: Array[Int] = Array(1, 20, 40)

scala> val intersectfunc = data1.intersection(data2)
intersectfunc: org.apache.spark.rdd.RDD[Int] = MapPartitionsRDD[12] at intersection at <console>:24

scala> intersectfunc.collect
res5: Array[Int] = Array(40, 20)
```

Cartesian function

val cartesianfunc = data1.cartesian(data2)

cartesianfunc.collect

```
scala> val cartesianfunc = data1.cartesian(data2)
cartesianfunc: org.apache.spark.rdd.RDD[(Int, Int)] = CartesianRDD[24] at cartesian at <console>:24

scala> cartesianfunc.collect
res22: Array[(Int, Int)] = Array((10,1), (10,20), (10,40), (20,1), (20,20), (20,40), (20,1), (20,20), (20,40), (40,1), (40,20), (40,40))
```

Sortbykey

val data = sc.parallelize(Seq(("C",3),("A",1),("D",4),("B",2),("E",5)))

data.collect

val sortfunc = data.sortByKey()

sortfunc.collect

```
scala> val data = sc.parallelize(Seq(("C",3),("A",1),("D",4),("B",2),("E",5)))
data: org.apache.spark.rdd.RDD[(String, Int)] = ParallelCollectionRDD[14] at parallelize at <console>:
23

scala> data.collect
res7: Array[(String, Int)] = Array((C,3), (A,1), (D,4), (B,2), (E,5))

scala> val sortfunc = data.sortByKey()
sortfunc: org.apache.spark.rdd.RDD[(String, Int)] = ShuffledRDD[17] at sortByKey at <console>:23

scala> sortfunc.collect
res8: Array[(String, Int)] = Array((A,1), (B,2), (C,3), (D,4), (E,5))
```

Spark groupByKey Function

val data = sc.parallelize(Seq(("C",3),("A",1),("B",4),("A",2),("B",5)))

data.collect

val groupfunc = data.groupByKey()

groupfunc.collect

```
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /___/ .__/\_,_/_/ /_/\_\   version 3.5.1
      /_/

Using Scala version 2.12.18 (Java HotSpot(TM) 64-Bit Server VM, Java 1.8.0_202)
Type in expressions to have them evaluated.
Type :help for more information.

scala> val data = sc.parallelize(Seq(("C",3),("A",1),("B",4),("A",2),("B",5)))
data: org.apache.spark.rdd.RDD[(String, Int)] = ParallelCollectionRDD[0] at parallelize at <console>:23

scala> data.collect
res0: Array[(String, Int)] = Array((C,3), (A,1), (B,4), (A,2), (B,5))

scala> val groupfunc = data.groupByKey()
groupfunc: org.apache.spark.rdd.RDD[(String, Iterable[Int])] = ShuffledRDD[1] at groupByKey at <console>:23

scala> groupfunc.collect
res1: Array[(String, Iterable[Int])] = Array((A,CompactBuffer(1, 2)), (B,CompactBuffer(4, 5)), (C,CompactBuffer(3)))
```

Spark reduceByKey Function

val reducefunc = data.reduceByKey((value, x) => (value + x))

reducefunc.collect

```
scala> val reducefunc = data.reduceByKey((value, x) => (value + x))
reducefunc: org.apache.spark.rdd.RDD[(String, Int)] = ShuffledRDD[2] at reduceByKey at <console>:23

scala> reducefunc.collect
res2: Array[(String, Int)] = Array((A,3), (B,9), (C,3))
```

Spark cogroup Function

val data1 = sc.parallelize(Seq(("A",1),("B",2),("C",3)))

data1.collect

val data2 = sc.parallelize(Seq(("B",4),("E",5)))

data2.collect

val cogroufunc = data1.cogroup(data2)

cogroupfunc.collect

```
scala> val data1 = sc.parallelize(Seq(("A",1),("B",2),("C",3)))
data1: org.apache.spark.rdd.RDD[(String, Int)] = ParallelCollectionRDD[3] at parallelize at <console>:23

scala> data1.collect
res3: Array[(String, Int)] = Array((A,1), (B,2), (C,3))

scala> val data2 = sc.parallelize(Seq(("B",4),("E",5)))
data2: org.apache.spark.rdd.RDD[(String, Int)] = ParallelCollectionRDD[4] at parallelize at <console>:23

scala> data2.collect
res4: Array[(String, Int)] = Array((B,4), (E,5))
```

```
scala> val cogroupfunc = data1.cogroup(data2)
cogroupfunc: org.apache.spark.rdd.RDD[(String, (Iterable[Int], Iterable[Int]))] = MapPartitionsRDD[6]
 at cogroup at <console>:24

scala> cogroupfunc.collect
res5: Array[(String, (Iterable[Int], Iterable[Int]))] = Array((A,(CompactBuffer(1),CompactBuffer())),
 (B,(CompactBuffer(2),CompactBuffer(4))), (C,(CompactBuffer(3),CompactBuffer())), (E,(CompactBuffer()
,CompactBuffer(5))))
```

Spark First Function

val data = sc.parallelize(List(10,20,30,40,50))

data.collect

val firstfunc = data.first()

```
scala> val data = sc.parallelize(List(10,20,30,40,50))
data: org.apache.spark.rdd.RDD[Int] = ParallelCollectionRDD[7] at parallelize at <console>:23

scala> data.collect
res6: Array[Int] = Array(10, 20, 30, 40, 50)

scala> val firstfunc = data.first()
firstfunc: Int = 10

scala>
```

Spark Take Function

val data = sc.parallelize(List(10,20,30,40,50))

data.collect

val takefunc = data.take(3)

```
scala> scala> val data = sc.parallelize(List(10,20,30,40,50))
```

```
scala> data.collect
res7: Array[Int] = Array(10, 20, 30, 40, 50)

scala> val takefunc = data.take(3)
takefunc: Array[Int] = Array(10, 20, 30)
```